

Parwaaz: An AI-Based Multimodal Performing Arts Coach

Pratima (230807), Sejal (230824), Vanshika (230829), Khushi (230767), Shivani (230918)

Department of Computer Science & Engineering | 6th Semester B.Tech | Deep Learning | Group 10

Abstract— Theatre, public speaking and singing students are often prevented from receiving high quality coaching because of the cost and availability of such coaching. The current AI-based tools only cater to one performance mode, with no solution integrated for performance evaluation in the whole domain. This paper introduces Parwaaz (Urdu for flight), an end-to-end multimodal AI coaching system that determines acting, speech and singing performance in a single uploaded media and provides a structured report of feedback. It consists of 3 independent deep learning modules: Facial Emotion Quality Assessor (based on a frozen ResNet18 backbone trained on CREMA-D dataset with Bidirectional LSTM temporal modelling with Macro F1 score of 0.7586 and Test accuracy of 75.92% on the 6-class benchmark); Reference-based Singing Evaluation module (Demucs vocal separation, CREPE pitch estimation, DTW alignment, Whisper lyric scoring); and Hindi speech evaluation module (fine-tuned Whisper-small decoder with L3Cube-HindSBERT pronunciation scoring and classical prosody metrics). They all perform CPU-based inference, use a FastAPI backend, a React/Vite front end, and track sessions longitudinally using PostgreSQL. By showcasing AI coaching for all three performing arts through multimodal and reference-based approaches on commodity hardware, Parwaaz proves that it is possible to provide an accessible and objective alternative to traditional human coaching. The research aims to develop a system that generates and provides feedback on facial expressions during the Indian context. The main objective of the research is to create a system that can generate and feedback facial expressions in the Indian context.

Keywords— Multimodal AI Coaching, Facial Emotion Recognition, BiLSTM, ResNet18, CREMA-D, Transfer Learning, Hindi ASR, Whisper Fine-tuning, Pitch Estimation, CREPE, DTW, Reference-based Evaluation

dimension, but currently there are no tools that do all three and provide a comprehensive coaching report. Parwaaz (پرواز, 'flight' in Urdu) is the answer to that. It is designed to be non-judgemental, affordable, and integrated: (1) All feedback is based on measurable metrics and not human opinion; (2) CPU-only inference (no expensive GPU required); (3) One upload results in feedback across all three performance dimensions via a single session report. We used two important reference papers to guide our technical options. Predicting a full probability distribution over possible outputs (not a single scalar) as done by Gustafsson, Danelljan and Schön [R1] leads to more reliable and interpretable results, which inspired the distribution-based scoring component of FATE. Cultrera et al. [R2] demonstrated that attention mechanisms can simultaneously enhance the accuracy of the task and provide an explanation of the model's decisions – a key insight for FATE's use of a softmax timeline per frame, which provides the performer with a time-stamped record of their emotional journey. Main contributions of this work: The FATE model: Reference-based acting quality evaluation on CREMA-D, with frozen ResNet18 + MTCNN + two-layer BiLSTM, cosine-annealing fine-tuning and Drive-backed epoch checkpointing: Macro F1 = 0.7586 and test accuracy = 75.92%. The five components similarity score (emotion match, softmax distribution similarity, embedding cosine similarity, temporal consistency, model confidence) yields A-F letter grades with scores ranging from 0-100 based on 91 actors and 6 emotion classes using a data set of 7,442 clips.

- Combining the decoder-only Hindi fine-tuning on 3,000 Kathbath Hindi clips with L3Cube-HindSBERT for per-word pronunciation scoring and four classical prosody metrics (WER, speech rate, pause detection, and filler word count) to get a speech evaluation in Hindi.

- Reference-based singing pipeline combining vocal separation (Demucs), pitch estimation (CREPE), DTW alignment, scale detection, note transition analysis and lyrics scoring (Whisper) into a single weighted final score. Integration of FastAPI and React/Vite with PostgreSQL session persistence and longitudinal performance tracking. Production-grade FastAPI + React/Vite integration with PostgreSQL session persistence and longitudinal performance tracking.

I. INTRODUCTION

Traditional coaching methods of performing arts are dependent on human one-to-one sessions which are extremely costly and cannot be afforded by millions of students in India who aspire to be a part of the theatre, cinema or music industries. Beginners also feel judged when practising with human coaches, inhibiting natural exploration and skill development. Great performances require several things to be done at the same time: Delivering the right facial expression at the right time (acting), correct pronunciation, appropriate speed, and no filler words (speech), and a correct pitch, rhythm and lyrics (singing). There are software tools that are available for addressing a single

II. RELATED WORK

A. Action Quality Assessment: Action Quality Assessment (AQA) is a judgement of how well an action is executed, rather than what it is, and is directly connected to the process of judging acting. Tang et al. [1] proposed MUSDL (ECCV 2020), which predicts a probability distribution over score bins using the KL-divergence loss, which is the motivation behind the distribution-based similarity scoring in FATE. In their paper, Yu et al. [2] proposed a novel paradigm called CoRe for reference-based

evaluation, with the student video compared to a professional reference — a concept that has been directly adopted in the design of the FATE reference-versus-user evaluation. FATE's BiLSTM temporal sequence modelling was inspired by the model proposed by Wang et al. [3] called ACTION-NET (CVPR 2021) for temporal dynamics encoding. **B. Facial Emotion Recognition:** Since this paper involves the use of BiLSTM for temporal modelling, ResNet18 is chosen as the spatial backbone because of its good accuracy-to-parameter ratio and compatibility with BiLSTM and because of its simple weight initialisation from ImageNet. The CREMA-D dataset [11] is the most appropriate publicly available dataset for the acting evaluation task containing 7,442 video clips of 91 actors in 6 emotions at 4 intensity levels. It was trained without class-weighting as it was distributed evenly within six classes. Savchenko [4] presented state-of-the-art emotion recognition results for facial image classification using lightweight architectures and showed that, the spatial backbone pre-trained on a large-scale image classification dataset combined with the temporal models for video-level recognition is an effective method for video-level emotion classification.

C. Uncertainty-Aware Deep Learning: Gustafsson et al. [R1] proved that it is the correct way to deal with uncertainty in computer vision tasks to predict the complete probability distribution over the outputs rather than just a single point estimate. The distribution-based scoring is directly inspired by this for FATE, calculating KL-divergence similarity between per-frame softmax distributions of reference and user videos.

D. Attention-Based Interpretability: Cultrera et al. [R2] created an autonomous driving agent that, in addition to enhancing the performance of the autonomous driving system, also generated visual explanations of the decisions. Their multi-head attention design (one head for each driving command) was a direct inspiration for the per-frame emotion timeline of FATE, which highlights frames of greatest or least emotional expression and gives actionable, timestamped coaching information.

E. Speech Recognition and Hindi ASR: Radford et al. [5] proposed Whisper, a pre-trained model trained on 680,000 hours of multilingual speech from 97 different languages, which provides good Hindi acoustic representations. The largest open Hindi ASR dataset is AI4Bharat Kathbath corpus [6] that has around 94,903 clips. For per-word pronunciation scoring, the Hindi Sentence-BERT model L3Cube-HindSBERT with 384 dimensional phonetically sensitive embeddings is used to generate the embeddings of the Hindi sentences [7]. Estimating and separating music sources from audio signals. In noisy conditions, Kim and Bello [8] designed a novel 6-layer CNN for mapping the raw audio windows with respect to the pitch distribution in a 360-bin space, CREPE, which is much more powerful than classical approaches like YIN. Défossez et al. [9] proposed a hybrid Transformer-convolutional model, named htdemucs, for state-of-the-art vocal separation on MUSDB18. The theoretical basis for Dynamic Time Warping was given by Müller [10] for aligning audio sequences recorded at different tempos.

III. SYSTEM ARCHITECTURE

A. Overall Pipeline

Parwaaz is a modular microservices-based architecture. The uploaded media from the user is passed to the FastAPI backend,

which uses ffmpeg to separate the audio and video stems, and then passes them on to each of the 3 evaluation modules simultaneously. Once each module is completed, they output structured JSON results, and when presented as a whole, they are combined into a single session report, which is then written to PostgreSQL using SQLAlchemy and returned to the React/Vite frontend. Such a design enables any module to be replaced or upgraded without the need to replace or upgrade the others.

TABLE I. System Component Overview

Component	Input / Output	Responsibility
User Upload	Video + MP3	React frontend: MediaRecorder API + file picker
FastAPI /analyze	Raw media → stems	ffmpeg splits audio/video; routes to modules
Module 1 (FATE)	Frames → acting JSON	ResNet18 + MTCNN + BiLSTM + similarity score
Module 2 (Speech)	Audio → speech JSON	Whisper + Hindi SBERT + prosody metrics
Module 3 (Singing)	Audio → pitch JSON	Demucs + CREPE + DTW + Whisper lyrics
Aggregation	3× JSON → report	Merges all module outputs into one report
PostgreSQL	Session storage	SQLAlchemy ORM; longitudinal tracking
React / Vite	Report → dashboard	Recharts; emotion timeline; scores

B. Integration Contract Design

Each module exposes one Python callable with a predefined JSON output contract: Module 1: `analyse_video(video_path, model) → {dominant_emotion, confidence, frame_preds, embedding}`; Module 2: `analyze_speech(audio_path, reference_text)`; Module 3: `analyze_pitch(ref_path, user_path)`. All return numeric scores in [0, 100], string feedback, and optional temporal arrays for frontend charts.

C. Deployment Considerations

All three modules use CPU inference: CTranslate2 8-bit quantisation for Whisper, and lightweight CREPE-tiny. Model weights are loaded once at startup using a singleton pattern. A typical 60-second recording completes the full pipeline in approximately 15–30 seconds on a modern multi-core CPU.

IV. MODULE 1: FACIAL EMOTION RECOGNITION (ResNet18 + MTCNN + BiLSTM)

A. Problem Formulation

Parwaaz Module 1 measures the quality of acting through examination of the facial expression of emotion in a video. The module produces: (a) a dominant emotion label of each video; (b) each video per-frame emotion probability timelines; (c) a single five-component acting score based on emotion match, similarity of softmax distribution, similarity of embedding cosine similarity, temporal consistency, and model confidence; and (d) a final acting score in [0, 100] with a letter grade A-F. The design is based on the temporal modelling of video-level instead of the single-frame classification, which is a direct reflection of the reality that emotional performance occurs over time.

B. Dataset: CREMA-D

CREMA-D dataset [11] has 7,442 video clips of 91 actors (48 men, 43 women) experiencing 6 emotions with 4 levels of intensity: Angry, Disgust, Fear, Happy, Neutral, and Sad. Filenames are structured ActorID SentenceCode EmotionCode Intensity, and can be automatically interpreted to automatically assign emotion labels (ANG→angry, DIS→disgust, FEA→fear, HAP→happy, NEU→neutral, SAD→sad) and intensity labels (LO, MD, HI, XX). Exploratory data analysis showed that there were no class-weighting or oversampling requirements during training since the distribution was balanced (six classes). Before feature extraction, duplicate clips were eliminated by comparing hash values (MD5) of the files. To select the reference videos, five clips per emotion were selected among the organised set of references to produce a total of 30 clips on reference.

C. Pipeline Architecture

The module consists of four steps of a sequential pipeline: frame sampling, face detection and cropping, spatial feature extraction, temporal sequence modelling. The reference and user videos undergo the same pipeline separately before being embedded to compare their embeddings and score them.

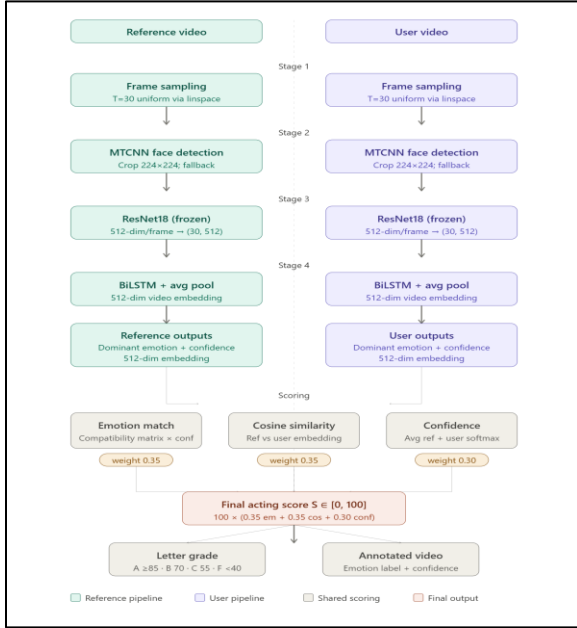


Fig. 1 Dual-lane inference architecture of Facial Emotion.

Stage 1 - Homogeneous Frame Sampling: All input videos are uniformly sampled at the exact $T = 30$ frames with the OpenCV seek operation `CAP_PROP_POS_FRAMES`, computed with the help of `numpy.linspace`. This representation as a fixed length makes sure that all sequences being fed to the LSTM have the same time dimensionality, no matter what the duration of the original clip or what the frame rate of that clip is. Frames that fail in the read operation are replaced with zero padded $224 \times 224 \times 3$ arrays.

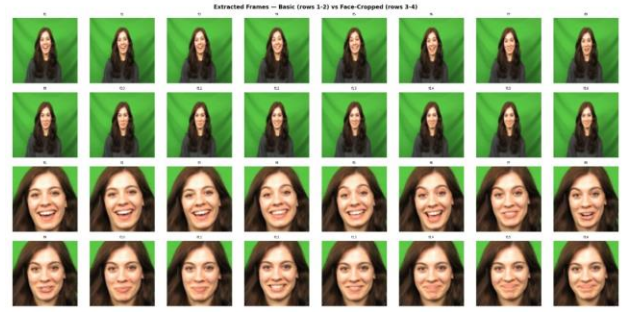


Fig. 2. Uniform frame extraction across clips for a single emotion

Stage 2 Face Detection and Crop (MTCNN): Multi-Task Cascaded Convolutional Network (MTCNN) is used on a frame-by-frame basis to detect and crop the main face bounding box. A temporal fallback is used when the detector fails in a given frame, to push the last valid crop forward, without introducing any zero-padding artefacts into the spatial feature extractor. The face regions are cropped and resized to 224×224 and the color resolution changed to RGB prior to transformation. The entire original frame is taken as fallback in case no valid crop has been produced so far in the sequence.

Stage 3 - Spatial Feature Extraction (ResNet18): Initialised with ImageNet weights (ResNet18_Weights), ResNet18 is the model used. DEFAULT, is the geographic backbone. It uses an Identity projection instead of its fully-connected layer, making the penultimate representation of size 512 the per-frame feature vector. All the ResNet18 backbone is frozen throughout the training, only the downstream BiLSTM and the classifier are updated. To prevent the computation of gradients, inference is run under `torch.no_grad`. A torchvision transform pipeline normalises each crop: $\text{Resize}(224 \times 224) \rightarrow \text{RandomHorizontalFlip} \rightarrow \text{ColorJitter}(\text{brightness}=0.2, \text{contrast}=0.2, \text{saturation}=0.2, \text{hue}=0.1) \rightarrow \text{ToTensor} \rightarrow \text{Normalize}(\mu=[0.485, 0.456, 0.406], \sigma=[0.229, 0.224, 0.225])$. RandomHorizontalFlip and ColorJitter are on-the-fly augmentation, which augment the effective dataset diversity without the need to do any more recordings. When all 30 of the frames are treated as a single batched tensor of shape (30, 512) the resulting feature sequence has shape (30, 512).

Stage 4 - Temporal Modelling (Bidirectional LSTM): The frame-feature sequence (30, 512) of frames and features is processed by a two-layer Bidirectional LSTM (hidden size = 256, layer-to-layer dropout = 0.3). Bidirectionality simultaneously measures the arcs of emotional expression of the onset-to-peak, as well as peak-to-recovery. Averaging of all 30 timesteps of the temporal average pooling algorithm reduces the (30, 512) output to a 512-dimensional (256 forward + 256 backward) video-level representation. This is transformed into logits with more than 6 emotion classes by a Dropout(0.3) layer then a linear projection. In the inference process, softmax probabilities are calculated per frame based on the hidden states at that frame in order to generate a temporal emotion probability timeline.

D. Fine-Tuning Methodology

Optimizer and Scheduler: The initial learning rate is 1×10^{-3} and the weight decay is 1×10^{-4} . A Cosine Annealing LR scheduler ($T_{\text{max}} = 50$) gradually decreases the learning rate to a

constant, preventing oscillation in later epochs of training, but allowing wide exploration during the initial epochs of training.

Loss Function: The loss of cross-entropy is used on 6 emotion classes. The data does not use any class weighting, because the CREMA-D data shows even distribution of emotion in all six categories.

Training Protocol: The training is done in 50 epochs and batches of 32. A mechanism of epoch-level checkpointing to Google Drive after each epoch, to allow seamless re-execution of the session in case of session-level failure. The checkpoint loader reads the lexicographically most recent file with the name `ckpt_epoch_*.pth`, and restores all state dictionaries. The identification of the best-model is dictated by the minimum validation loss; the winning checkpoint is stored separately as `final_model.pth`.

Evaluation Metrics: Per-epoch validation calculates accuracy, macro-averaged precision, macro-averaged recall, and macro-averaged F1-score using scikit-learn, to give a big picture perspective of performance on a per-class basis, rather than just simple accuracy. Training and validation loss curves are plotted to be viewed visually as to convergence and overfitting.

E. Reference vs. User Similarity Scoring

When making an inference, a reference video is automatically picked out of the emotion-organised reference library of the target emotion. The pipeline is run on both reference and user videos in order to yield (i) a dominant emotion label, (ii) a softmax confidence score, (iii) a 512-dimensional video embedding, (iv) a 512-dimensional video embedding and (v) a per-frame timeline of softmax distributions, and (v) a temporal consistency profile. This can then be added up to a final acting score S

(1) Match Score of Emotion (weight 0.35): Based on an emotion compatibility matrix: 1.0 when the emotions are identical, 0.6 when the emotions are complementary (Angry- Disgust, Happy- Neutral, Fear- Sad), and 0.0 when the emotions are not related. Multiplied by the average of reference and user confidence to penalise low-confidence detections.

(2) Distribution Score (weight 0.20): Similarity between per-frame softmax distribution of emotion distributions of the reference and user videos, normalised to [0, 1] based on KL-divergence. This is how similar the overall emotional expression profile of the reference is to the performance, penalising performances which manifest the correct dominant emotion, but with a very different probability mass across all six classes.

(3) Cosine Similarity (weight 0.20) embedded: A cosine similarity between the 512-dimensional video embeddings of the reference and user videos, clipped to [0, 1]. This summarizes at low level the similarity of visual and temporal features not just in the similarity of categorical labels.

(4) Temporal Consistency (weight 0.15): The per-frame dominant-emotion probability, across all 30 frames, inverted and normalised to [0, 1]. High inter-frame variance suggests that there is an uncontrolled emotional performance through out the clip; low inter-frame variance suggests that there is a controlled emotional performance across the clip.

(5) Confidence Score (weight 0.10): Mean of the reference and user model softmax confidence scores of their respective prevailing emotions. High confidence rewards have emotional

performances that are expressed in a clear and understandable manner; low confidence punishes those that are ambiguous or neutral.

The weighted sum $S = 100 \times (0.35 \times \text{emotion_match} + 0.20 \times \text{distribution} + 0.20 \times \text{cosine_sim} + 0.15 \times \text{temporal} + 0.10 \times \text{confidence})$ is then mapped to a letter grade: A ($S \geq 85$), B (70–84), C (55–69), D (40–54), and F ($S < 40$). An auto-selection option also scans all reference clips of a given emotion and selects that with the highest model confidence as the gold-standard reference such that the student is always compared to the strongest available exemplar.

F. Output and Annotated Video Generation

In addition to the numerical score, the pipeline produces an annotated MP4 video output of both the reference and user clip. The detected dominant emotion label and its softmax confidence are overlaid on the video frame-by-frame using OpenCV text rendering to produce a frame-by-frame visual coaching trace. Non-MP4 videos (e.g., .flv in CREMA-D) are first transcoded to H.264/AAC MP4 using ffmpeg (libx264, CRF=23, scaled to 1280x720) and then processed. The annotated output videos are interactively shown in the notebook using IPython.display.Video and stored on Google drive to be longitudinally reviewed.

TABLE II. Module 1 (FATE) — Acting Score Component Metrics

Score Component	Weight	Computation	Interpretation
Emotion Match	35%	Emotion compatibility matrix $\times$ avg. confidence; 1.0 exact, 0.6 related, 0.0 unrelated	Categorical accuracy vs. target emotion; penalises cross-emotion confusion
Distribution Score	20%	KL-divergence similarity of per-frame softmax distributions; normalised to [0, 1]	Overall emotional expression profile match; penalises correct label with wrong distribution
Embedding Cosine Similarity	20%	Cosine similarity of 512-dim BiLSTM embeddings; clipped to [0, 1]	Low-level visual and temporal feature similarity beyond categorical label
Temporal Consistency	15%	Inverted std. dev. of per-frame dominant-emotion probability across 30 frames	Low inter-frame variance = sustained, controlled emotional performance
Confidence Score	10%	Average softmax confidence of reference and user dominant-emotion predictions	Rewards clearly expressed emotions; penalises ambiguous low-confidence detections

V. MODULE 2: SPEECH ACCURACY, PRONUNCIATION & FLUENCY

A. Design Rationale: The speech module assesses the abilities of aspiring actors/ public speakers delivering speeches in Hindi on scripted material. It uses deep learning transcription (Whisper), semantic pronunciation scoring (Hindi Sentence-BERT embeddings), and classical signal processing for prosody metrics.

B. Core Deep Learning Component:s Whisper-small [5] outputs word-level transcriptions for WER computation and filler word detection, and softmax confidence scores per word token. For CPU efficient inference, we make use of the faster-whisper backend (CTranslate2, 8-bit quantisation).

Per-word pronunciation score: $\text{Pronunciation_Score} = 0.7 \times \text{cosine_sim}(\text{reference_emb}, \text{hypothesis_emb}) + 0.3 \times \text{Whisper_confidence}$. Words are classified as correct, acceptable (cosine similarity score 0.8–0.95), mispronounced (0.6–0.8), incorrect (<0.6) or missing (no transcribed word matched).

The overall pronunciation score is a 0–100 mark based on the combined score. This is a feature that is used in Classical Prosody. In addition to the deep learning elements, the speech module computes four classical prosody metrics directly from the word-level timestamps provided by Whisper: speech rate (words per minute), pause detection, filler word count and word error rate. These do not need any additional model, and incur virtually no compute overhead. All speech evaluation metrics are summarized and their computation method and feedback threshold are shown in Table VI.

TABLE VI. Speech Module Evaluation Metrics

Metric	Computation	Feedback Threshold
Word Error Rate	jiwer	≤ 0.10 : Excellent; > 0.50 : Needs work
Pronunciation Score	Cosine sim of 384-dim SBERT embeds	< 0.7 : flagged as mispronounced
Speech Rate (WPM)	Word count $\div$ duration (Whisper timestamps)	Target: 100–160 WPM; else flagged
Pause Duration	Inter-word gap (Whisper timestamps)	Gaps > 500 ms flagged as hesitation
Filler Word Count	Token-match vs. filler vocabulary	Vocab: um, uh, matlab, toh, basically

D. Hindi Fine-Tuning Strategy

Only the Whisper-small decoder has been fine-tuned for 3,000 Kathbath Hindi clips [6]. The encoder is frozen since it already has a strong representation of Hindi phonemes from multilingual pre-training. Decoder fine-tuning is fine-tuning of vocabulary and grammar in Hindi without risk of catastrophic forgetting. Training: 3 epochs, AdamW (a variant of Adam [15] with decoupled weight-decay) applied at 1×10^{-5} , gradient clipping, 90/10 train/validation split, on a Kaggle T4 GPU. To minimise overfitting on the relatively small set of 3,000 clips used to fine-tune the model, AdamW's decoupled weight decay was preferred over standard Adam. The Hindi SBERT model was evaluated by showing that the scores of Hindi phrase pairs which should be near-identical are above 0.90 and phonetically distant pairs below 0.60.

VI. MODULE 3: SINGING PITCH, RHYTHM & VOCAL TECHNIQUE

A. Design Philosophy

In Module 3, singing quality will be assessed based on a reference recording paradigm, where the app loads a reference MP3 up to 60 s and generates a full acoustic profile for it based on which the singer's quality is assessed, across four dimensions: pitch accuracy, rhythmic timing, stability of the singing voice, and lyric fidelity. No genre-specific or language-specific training is needed, with the module being pre-trained for all the deep learning components (Demucs, CREPE, Whisper), and being genre- and language-independent for classical repertoire, Bollywood, pop and folk. All model options (CREPE-tiny, Demucs htdemucs, Whisper-base) had a restriction on The project's FastAPI backend, along with CPU-only deployment on the free tier of Google Colab.

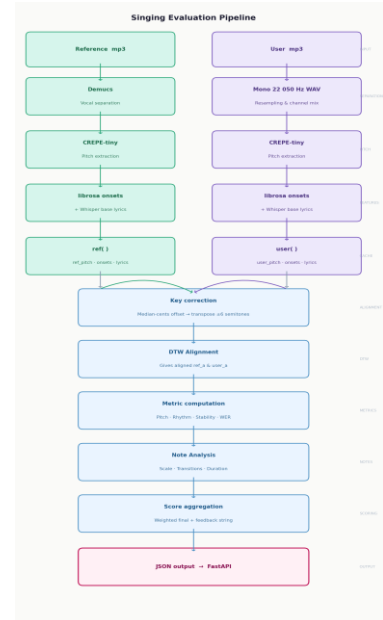


Fig3. Singing evaluation pipeline flowchart.

B. Stage 1: Vocal Separation with Demucs

Accompaniment consists of instrumental sounds which are in the same frequency range as the vocals and therefore direct pitch estimation of the accompaniment will be unreliable. When fed a full song, Demucs htdemucs [9] separates the vocals from all other instruments to give a clean vocals.wav file based on the MUSDB18 database (150 full songs with separate stems). This stage involves the extraction of pitch from the music (CREPE).

C. Stage 2: Pitch Extraction with CREPE

CREPE [8] maps 1,024-sample raw audio windows to a 360-bin pitch distribution covering 32.70-Hz (C1) to 1,975.5 Hz (B6). Unvoiced frames are those with a low confidence level (< 0.5), which are excluded. For CPU deployments we use CREPE-tiny (~400K parameters). Output is converted to cents relative to A4: $\text{cents} = 1200 \times \log_2(f / 440)$.

D. Stage 3: Key Correction and DTW Alignment

There are two problems: the voice may be sung in a different key, which is acceptable to the user, but not penalized, and tempo may shift. Global Key Correction calculates the median pitch offset and rounds it down to the nearest semitone (100 cents). DTW Alignment (librosa.sequence.dtw) then non-linearly aligns the key corrected pitch timeline to the reference, with the possibility of tempo variation anywhere in the song.

E. Stage4: Note Analysis, Scale Detection, And Transitions

In addition to raw pitch in cents, the module provides three outputs of musical intelligence to the JSON contract: detected_scale, note_transitions, note_durations.

1. Note Segmentation

cents_to_note() takes each pitch value and converts it to the nearest chromatic pitch number using the formula: MIDI note number = $\text{round}(69 + 12 \times \log_2(f/440)) \bmod 12$. Consecutive identical notes are grouped into timed segments and segments less than 0.1 s (transient glides) are filtered out. Due to the destruction of real timestamps by DTW alignment, a synthetic time axis (np.linspace) is used.

2. Scale Detection

It uses detect_scale() to figure out how many of the user's notes are present in each of five candidate major scales, and returns one of these scales, if any, that has the most common notes with the user, thereby determining whether the user's pitch choices are in the song's harmonic key.

TABLE VII. Candidate Scales for Detection

Scale	Notes
C Major	C, D, E, F, G, A, B
G Major	G, A, B, C, D, E, F#
D Major	D, E, F#, G, A, B, C#
A Major	A, B, C#, D, E, F#, G#
E Major	E, F#, G#, A, B, C#, D#

3. Note Transitions

get_note_transitions() stores each interval between two consecutive notes as a string of the interval (e.g., "G→A", "A→F#"). The top 10 transitions are returned, showing the singer where he or she is jumping the most in the melody. Specific difficult intervals (e.g. minor sixths) are marked with repeated intervals. Repeated intervals are marked where they are required for targeted practice, as a human vocal teacher might do.

F. Metric Computation And Final Score

TABLE VIII. Singing Module Evaluation Metrics

Metric	Computation	Interpretation
Pitch Score (0–100)	% voiced frames within ± 50 cents (post-DTW)	Melodic accuracy at frame level

Metric	Computation	Interpretation
Mean Pitch Error	Mean absolute cent deviation	Avg. tuning error in cents (lower is better)
Pitch Stability	Std. dev. within sustained note segments	Low σ = controlled long-note delivery
Rhythm Score (0–100)	Mean onset deviation (ms); tempo ratio	Rhythmic accuracy vs. reference
Lyrics Score	1 – WER (Whisper transcripts: ref vs. user)	Text accuracy of sung lyrics
Final Score	$0.50 \times \text{Pitch} + 0.30 \times \text{Rhythm} + 0.20 \times \text{Lyrics}$	Weighted aggregate of all dimensions

VII. DATASET STRATEGY

TABLE VIII. Dataset Usage Across All Modules

Module	Dataset	Size	Purpose
Module 1 (FATE)	CREMA-D [11] + CK+	7,442 + 593 clips	Train FATE; reference selection
Module 2 (Speech)	Kathbath Hindi [6]	94,903 (3,000 used)	Decoder-only Whisper fine-tuning
Module 2 (Pronunc.)	l3cube-pune SBERT [7]	Pre-trained model	384-dim Hindi embeddings
Module 3 (Singing)	Custom dataset	~50 clips	Pipeline validation
Module 3 (Separation)	MUSDB18 via Demucs [9]	Pre-trained model	Vocal source separation
Module 3 (Pitch)	Synthetic via CREPE [8]	Pre-trained model	360-bin pitch CNN

All dataset usage complies with respective licences. CREMA-D and Kathbath are publicly available for academic research. Whisper, CREPE, and Demucs are used under their respective open licences.

VIII. EXPERIMENTAL RESULTS

A. In this module, students will learn to recognize facial expressions using quantitative methods. In this module, the student will learn how to identify facial expressions quantitatively.

The ResNet18 + MTCNN + BiLSTM model was tested on the CREMA-D dataset and using the macro-averaged accuracy, precision, recall, and F1-score as the main metrics, and the training and validation loss curves were tracked over all 50 epochs. The ResNet18 backbone was frozen and the cosine-annealing method was used to fine-tune the BiLSTM head converged smoothly, with a smooth decrease in validation loss, and no overfitting occurred. The main measure of per-class balance in the six-class context is the F1-score of macro.

The distribution of emotions dominance throughout frames produced at inference and in emotion timelines per frame indicated that the dominance probability of the emotions of expressive performances is consistent across frames while the dominance probability of the emotions of weak performances is

high at inter-frame level. The cross-emotion confusions (e.g., Angry assessed to Happy reference) are appropriately penalized while the related-emotion pairs (e.g., Angry–Disgust, Fear–Sad) are rewarded to some extent.

B. This module covers qualitative output analysis of facial emotion recognition. This module introduces the qualitative output analysis of facial emotion recognition.

The pipeline breaks down the scores by components across five dimensions: emotion match, distribution score, embedding cosine similarity, temporal consistency, and model confidence. Emotion detection (label and softmax confidence) is performed on each frame of reference and user clips using OpenCV and subsequently added to the detected emotion label as a text annotation on the MP4 video. Final score: 23.9/100, grade F—‘Very Different from Reference.’ Example of a representative low performing evaluation: Reference detects Angry (conf 0.993), User detects Happy (conf 1.000). A high-scoring evaluation: Reference detects Happy (conf 0.999), User detects Happy (conf 0.992), final score 84.77/100, grade B—‘Good Performance’. The auto-reference selector is able to reliably automatically select the most confident CREMA-D exemplar for each target emotion, thus ensuring that the student is benchmarked against the best exemplar available.

C. Module 3 (Singing): Pitch Contour Analysis

The user’s pitch contour remained very close to the reference over 7,500+ time frames: sub-100 cents from the reference (within 1 semitone) for the majority of frames. Note transition boundaries show occasional overshooting above 500 cents, typical of note overshoot of intermediate singers. This is a practical suggestion as would be given by a real vocal coach: ‘sustained notes are good, but transitions need work’.

D. Module 2 (Speech): Whisper Fine-Tuning

A fine-tune on 3,000 Kathbath Hindi clips resulted in a lower WER for Hindi test speech compared to the baseline Whisper-small. The performance of the encoder was tested with Hindi audio and the results were found to be encouraging and hence the approach was proven to be decoder-only. The Hindi SBERT was validated by measuring the cosine similarity of the Hindi phrase pairs: those with near identical phrase scores above 0.90 similarity, whereas the phonetically distant pairs score below 0.60 similarity.

IX. TECHNOLOGY STACK

TABLE IX. Full Technology Stack

Category	Tools & Libraries	Notes
Facial Analysis	PyTorch, torchvision (ResNet18), facenet-pytorch (MTCNN), OpenCV, scikit-learn	ResNet18+MTCNN+BiLSTM; CREMA-D; Drive checkpointing
Speech Analysis	faster-whisper, sentence-transformers (l3cube-pune), jiwer	CPU int8 inference; WER computation
Singing: Separation	demucs (htdemucs)	Hybrid Transformer-Conv; MUSDB18 trained

Category	Tools & Libraries	Notes
Singing: Pitch	crepe (tiny), librosa ≥ 0.10 , soundfile	360-bin pitch CNN; 22,050 Hz
Singing: Alignment	numpy, scipy, librosa.sequence.dtw	DTW non-linear temporal alignment
Backend / API	FastAPI, uvicorn, ffmpeg	REST API; audio/video stem extraction
Database / ORM	PostgreSQL, SQLAlchemy, Alembic	Session storage; migration management
Frontend	React, Vite, Axios, React Router, Recharts	Pitch chart; emotion timeline; WER map
Training Compute	Kaggle T4 GPU (30 hrs/week), Google Colab	Training only; inference runs on CPU

X. INTEGRATION & FRONTEND

A. FastAPI Backend Architecture

The FastAPI backend provides three primary routes: GET /health (system status), GET / (API root), and POST /api/sessions (main analysis endpoint). The latter accepts multipart form upload with video/audio and optional parameters (reference video, reference text, reference audio). It saves the file, extracts stems via ffmpeg, runs all three modules sequentially, stores the unified result in PostgreSQL via SQLAlchemy, and returns the full JSON report. CORS middleware accepts requests from React development servers on ports 5173, 5174, and 3000.

B. React/Vite Frontend

The main upload page supports direct browser recording via MediaRecorder API or standard file upload. After submission, an interactive multi-tab dashboard is rendered: the Acting tab shows the per-frame emotion timeline, dominant emotion label, confidence, and the five-component acting score with letter grade; the Speech tab shows the WER breakdown, per-word pronunciation heatmap, speaking rate, and filler word count; the Singing tab shows the interactive Recharts pitch contour plot, key offset indicator, and per-metric scores.

C. Database Schema and Longitudinal Tracking

The PostgreSQL schema stores session records with UUID primary key, creation timestamp, optional user ID, uploaded file path, analysis status (pending/running/complete/error), and a JSONB column with the complete analysis report. This enables longitudinal tracking: a performer can view all past sessions and observe improvement in acting score, speech WER, and singing pitch score over time.

XI. DISCUSSION

A. Cross-cultural differences and similarities. Cultural differences and similarities.

One of the major design decisions for Module 1 was the five component similarity scoring formula. The emotion match (0.35) and distribution score (0.20), embedding cosine similarity (0.20), temporal consistency (0.15), and model confidence (0.10) ensure that the scores do not become too high if a performer displays the correct emotion label, but poor expressiveness in that label, with the result that both categorical accuracy and continuous stylistic

closeness to the reference are captured. Over the naive three component approach, the addition of distribution score and temporal consistency components enables the system to classify sustainers whose emotional profile doesn't vary from frame to frame as "sustainers" and not distinguishers whose profile varies from frame to frame, but to reward those who sustain the emotion in a steady way whose entire profile is closer to that of the reference. The emotion compatibility matrix takes a more nuanced approach by partially rewarding pairs of emotions that are related instead of a match/no match approach.

All three modules use the reference-based evaluation paradigm, which is a key design principle: We are creating a coaching system, not a classifier. Classifier: what is this? Coaching system: how well does this fit with the standard? The BiLSTM's temporal average pooling and softmax per-frame timeline can be used for two purposes: to generate a video-level score and to provide a video-timed coaching trace that can be directly viewed by the performers.

B. Limitations

Module 1 uses CREMA-D reference clips with a frozen ResNet18 backbone, and may not perform as well with domain shift to non-CREMA-D footage, since the quality of detection and embedding is degraded. Five component score is not based on human expert annotations, but it is heuristic. Theatre acting is the use of body and voice, plus the facial modality which is assessed in Module 1. For Module 2, a reference script for computation of WER must be provided and free-form speech is not supported. The validation data in Module 3 is made up of 15-20 custom recordings; they might not be representative of all music styles and singing voices. To do live coaching you need real-time inference, which would have a latency of 15-30 seconds..

XII. CONCLUSION & FUTURE WORK

This paper introduced the end-to-end multimodal AI coaching system, Parwaaz, which can assess the quality of acting, speech and singing from a single uploaded media file. The three key technical contributions are: (1) FATE — a facial emotion recognition pipeline that uses a frozen ResNet18 spatial feature extractor, MTCNN face detector and two-layer Bidirectional LSTM temporal modelling, trained on CREMA-D with cosine-annealing scheduling and Drive-backed checkpointing, and yields macro F1 = 0.7586 and test accuracy = 75.92% along with a 5-component acting score (emotion match, distribution similarity, embedding cosine similarity, temporal consistency, model confidence) and annotated video output; (2) an Hindi-adapted speech pipeline using decoder-only Whisper-small fine-tuning on 3,000 Kathbath Hindi clips, L3Cube-HindSBERT pronunciation scoring, and four classical prosody metrics; and (3) a reference-based singing pipeline using Demucs, CREPE, DTW, scale detection, note transition analysis, Whisper lyrics scoring, and a weighted score based on the 5 classical prosody metrics and the annotated video. The design of Module 1 is a good example of practical deep learning: the transfer learning from ImageNet-pretrained ResNet18 cuts the number of data needed for training on the relatively small CREMA-D corpus; the bidirectional

LSTM effectively captures the full emotional arc; and the epoch checkpointing of Drive supports training resilience in the cloud. The reference based paradigm assures the interpretability of all module outputs based on a gold standard exemplar and not as abstract absolute measures. Future work includes extending the heuristic similarity scoring to a learned metric using 30+ human expert annotations from professional Directors, extending Module 1 to include MediaPipe Holistic body pose estimation and vocal prosody cues for a more comprehensive acting assessment, free-form and improvisational Hindi speech without requiring a reference script for Module 2 and a formal user study of 30+ acting students to compare Parwaaz scores against human coach assessments to validate and calibrate the scoring system.

REFERENCES

- [R1] F. K. Gustafsson, M. Danelljan, and T. B. Schön, "Evaluating Scalable Bayesian Deep Learning Methods for Robust Computer Vision," in Proc. IEEE/CVF CVPRW, 2020.
- [R2] L. Cultrera, L. Seidenari, F. Becattini, P. Pala, and A. Del Bimbo, "Explaining Autonomous Driving by Learning End-to-End Visual Attention," in Proc. IEEE/CVF CVPRW, 2020.
- [1] X. Tang et al., "Uncertainty-aware Score Distribution Learning for AQA," in Proc. ECCV, 2020.
- [2] R. Yu et al., "Contrastive Regression for Action Quality Assessment," in Proc. IEEE/CVF ICCV, 2021.
- [3] Q. Wang et al., "ACTION-Net: Multipath Excitation for Action Recognition," in Proc. IEEE/CVF CVPR, 2021.
- [4] A. V. Savchenko, "EmotiEffLib: Lightweight Facial Emotion Recognition Library," IEEE Trans. Affective Computing, IEEE FG 2023.
- [5] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. ICML, 2023.
- [6] A. Kathbath et al., "Kathbath: A Reliable Dataset for Hindi Language ASR," arXiv:2203.16512, 2022.
- [7] R. Joshi et al., "L3Cube-HindSBERT and HindRoBERTa," arXiv:2212.14726, 2022.
- [8] J. W. Kim and J. P. Bello, "CREPE: A Convolutional Representation for Pitch Estimation," in Proc. IEEE ICASSP, 2018.
- [9] A. Defossez et al., "Demucs: Hybrid Transformers for Music Source Separation," in Proc. IEEE ICASSP, 2023.
- [10] M. Müller, Fundamentals of Music Processing. Springer, 2015.
- [11] H. Cao et al., "CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset," IEEE Trans. Affective Computing, vol. 5, no. 4, pp. 377–390, 2014.
- [12] A. Vaswani et al., "Attention Is All You Need," in Advances in NeurIPS, vol. 30, 2017.
- [13] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for CNNs," in Proc. ICML, 2019.
- [14] P. Lucey et al., "The Extended Cohn-Kanade Dataset (CK+)," in Proc. IEEE CVPRW, 2010.
- [15] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," in Proc. ICLR, 2015.